

Steganography and Hidden Data Detection

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

5 September 2026

LAB OVERVIEW

In this laboratory exercise, I explored digital steganography and the forensic analysis of hidden data. I learned how information can be concealed within an ordinary image and examined techniques such as Least Significant Bit (LSB) steganography. I used Steghide to embed and extract a secret file, then compared the original and stego images using their file sizes, SHA-256 hashes, hexadecimal data with `xxd`, and readable strings. I also performed a controlled StegSeek dictionary test to understand password-based recovery of hidden information. Through these practical activities, I gained a better understanding of how hidden data can be identified, extracted, verified, and properly documented during a digital forensic investigation.

OBJECTIVES

- Differentiate between steganography and encryption and explain their respective purposes in data protection.
- Describe common data-hiding techniques, including insertion, substitution, and Least Significant Bit (LSB) techniques.
- Prepare a carrier image and secret evidence file for a controlled steganographic experiment.
- Use Steghide to embed and extract concealed data from a carrier image.
- Calculate and compare cryptographic hashes to verify the integrity of the original and extracted evidence.
- Perform a controlled password dictionary test using StegSeek to demonstrate password-based recovery of hidden data.
- Document the procedures, commands, results, and observations in a clear and repeatable forensic report.

TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Github, Git
- Md5sum and sha256sum
- Steghide, stegseek, xxd

METHODOLOGY

The first step is to download the required bmp image to run the lab

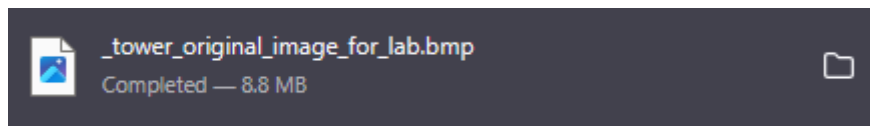


Fig 1.1 Downloaded bmp image

Part A - Prepare and Preserve the Evidence

We then create a lab environment to preserve the evidence by creating a dedicated working folder

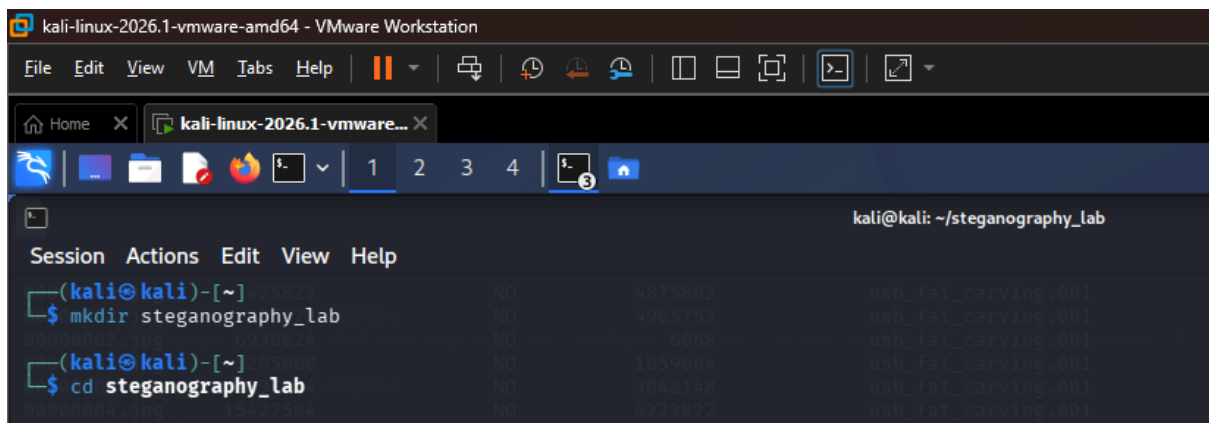


Fig 1.2 Created a dedicated working folder

Next, we download the bitmap image in the dedicated folder and then we confirmed the file type

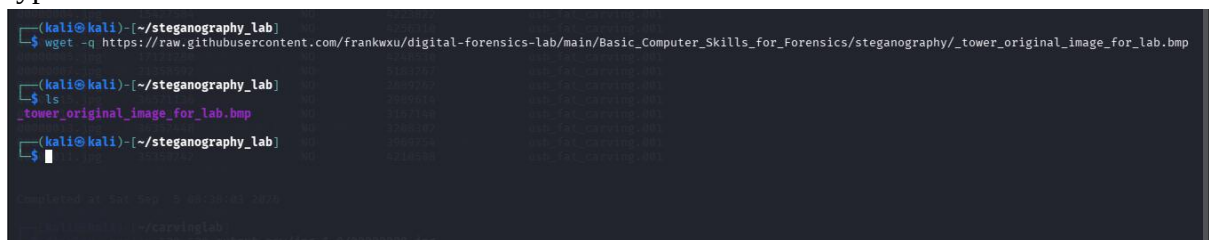


Fig 1.3 Downloaded the bmp image and checked the file type

Next, we calculate the sha256sum hash value

```

(kali@kali)-[~/steganography_lab]
$ sha256sum _tower_original_image_for_lab.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db _tower_original_image_for_lab.bmp

(kali@kali)-[~/steganography_lab]
$

```

Fig 1.4 sha256sum value of the bmp image

Now I created a copy of the evidence to preserve it

```

(kali@kali)-[~/steganography_lab]
$ cp _tower_original_image_for_lab.bmp _tower_image_for_lab.bmp

(kali@kali)-[~/steganography_lab]
$ ls
_tower_image_for_lab.bmp _tower_original_image_for_lab.bmp

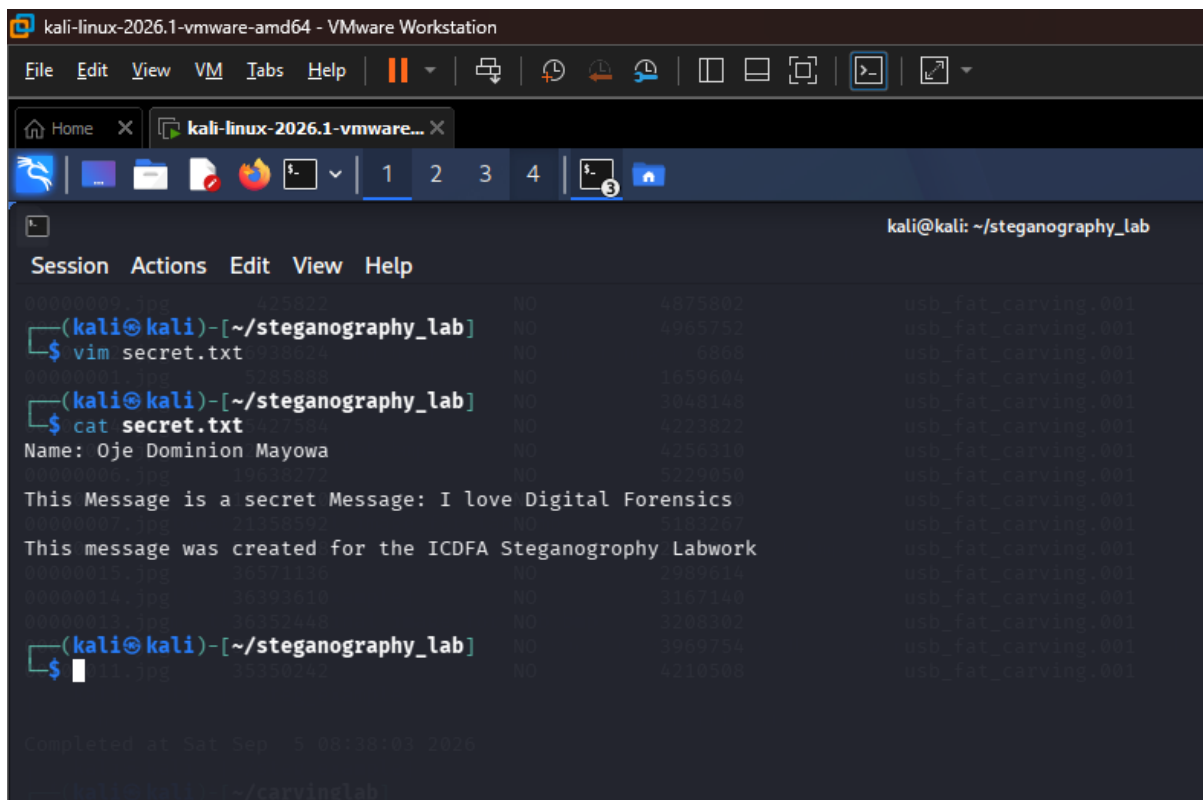
(kali@kali)-[~/steganography_lab]
$

```

Fig 1.5 Preserving evidence by creating a copy of the original

Part B - Create the Secret Evidence File

Now we create the secret evidence file and include name, the secret message and authorize the message



```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware... X
kali@kali: ~/steganography_lab

Session Actions Edit View Help
00000008.jpg 4258633 NO 4875862 usb_fat_carving.001
(kali@kali)-[~/steganography_lab] NO 4963752 usb_fat_carving.001
$ vim secret.txt NO 6868 usb_fat_carving.001
00000011.jpg 5285888 NO 1659604 usb_fat_carving.001
(kali@kali)-[~/steganography_lab] NO 3048148 usb_fat_carving.001
$ cat secret.txt NO 4223822 usb_fat_carving.001
Name: Oje Dominion Mayowa NO 42586310 usb_fat_carving.001
00000006.jpg 19836372 NO 5229050 usb_fat_carving.001
This Message is a secret Message: I love Digital Forensics NO 5183367 usb_fat_carving.001
00000007.jpg 21358582 NO 5183367 usb_fat_carving.001
This message was created for the ICDFA Steganography Labwork NO 2989614 usb_fat_carving.001
00000015.jpg 36371136 NO 3167148 usb_fat_carving.001
00000014.jpg 36393610 NO 3208302 usb_fat_carving.001
00000013.jpg 36357668 NO 3969754 usb_fat_carving.001
(kali@kali)-[~/steganography_lab] NO 4218506 usb_fat_carving.001
$
11.jpg 35358242 NO
Completed at Sat Sep 5 08:38:03 2026
kali@kali: ~/carvinglab

```

Fig 1.6 Secret Message Created

Next, the sha256sum hash value was calculated for the file

```
(kali㉿kali)-[~/steganography_lab]
└─$ sha256sum secret.txt
1bc89f1c35a4a03e51ca13c254fe1937abda936a9ffd1f371ed00b4059e3a97a  secret.txt

(kali㉿kali)-[~/steganography_lab]
└─$
```

Fig 1.7 Hash value calculated for the secret file

Part C - Embed the Hidden File with Steghide

Now we embed the secret file into the bmp image and then the hash value for the new stego image was gotten and it is seen evidently that the hash value differs from the initial hash value of the original bmp image

```
(kali㉿kali)-[~/steganography_lab]
└─$ steghide embed -ef secret.txt -cf _tower_image_for_lab.bmp -sf tower_stego_lab4.bmp -p 1234
embedding "secret.txt" in "_tower_image_for_lab.bmp" ... done
writing stego file "tower_stego_lab4.bmp" ... done

(kali㉿kali)-[~/steganography_lab]
└─$ sha256sum tower_stego_lab4.bmp
4d89ebd54a98e981b416a692da1470a7579e3e9be9072751bd43a26c4e777a45  tower_stego_lab4.bmp

(kali㉿kali)-[~/steganography_lab]
└─$
```

Fig 1.8 Embedded secret text into the bmp file and got the hash value of the new stego image

Part D - Extract and Verify the Hidden Evidence

Extracted the hidden text inside the stego_image into a file called extracted.txt

```
(kali㉿kali)-[~/steganography_lab]
└─$ mkdir -p extracted
└─$ steghide extract -sf tower_stego_lab4.bmp -xf extracted/extracted_secret.txt -p 1234
wrote extracted data to "extracted/extracted_secret.txt".

(kali㉿kali)-[~/steganography_lab]
└─$
```

Fig 1.9 extracted Secret text into a file

Then for us to compare the sha256sum values for both the original secret.txt file and the extracted.txt file we used the **diff** command.

I put the hashed code of the secret.txt file into a original_hash.txt file and the hashed code of the extracted file into an extracted_hash.txt file and the result shows that both files have the same sha256sum value as stored in their respective files.

```
(kali@kali)-[~/steganography_lab]
$ sha256sum secret.txt > original_hash.txt original_image_for_lab.bmp 1234
original_image_for_lab.bmp... done

(kali@kali)-[~/steganography_lab]
$ sha256sum extracted/extracted_secret.txt > extracted_hash.txt
extracted_image_for_lab.bmp

(kali@kali)-[~/steganography_lab]
$ diff original_hash.txt extracted_hash.txt
1c1
< 1bc89f1c35a4a03e51ca13c254fe1937abda936a9ffd1f371ed00b4059e3a97a secret.txt
> 1bc89f1c35a4a03e51ca13c254fe1937abda936a9ffd1f371ed00b4059e3a97a extracted/extracted_secret.txt

(kali@kali)-[~/steganography_lab]
$
```

Fig 1.8 Embedded secret text into the bmp file and got the hash value of the new stego image

Part E - Compare Original and Stego Images

Comparison between the Original and the Stego Images,

1. **SIZE:** Both the original and the stego image has the same size

```
(kali@kali)-[~/steganography_lab]
$ ls
extracted extracted_hash.txt original_hash.txt secret.txt _tower_image_for_lab.bmp _tower_original_image_for_lab.bmp tower_stego_lab4.bmp

(kali@kali)-[~/steganography_lab]
$ ls -lh _tower_original_image_for_lab.bmp tower_stego_lab4.bmp
-rw-rw-r-- 1 kali kali 8.9M Sep  5 09:55 _tower_original_image_for_lab.bmp
-rw-rw-r-- 1 kali kali 8.9M Sep  5 10:31 tower_stego_lab4.bmp

(kali@kali)-[~/steganography_lab]
$
```

Fig 1.9 The comparison between the file sizes

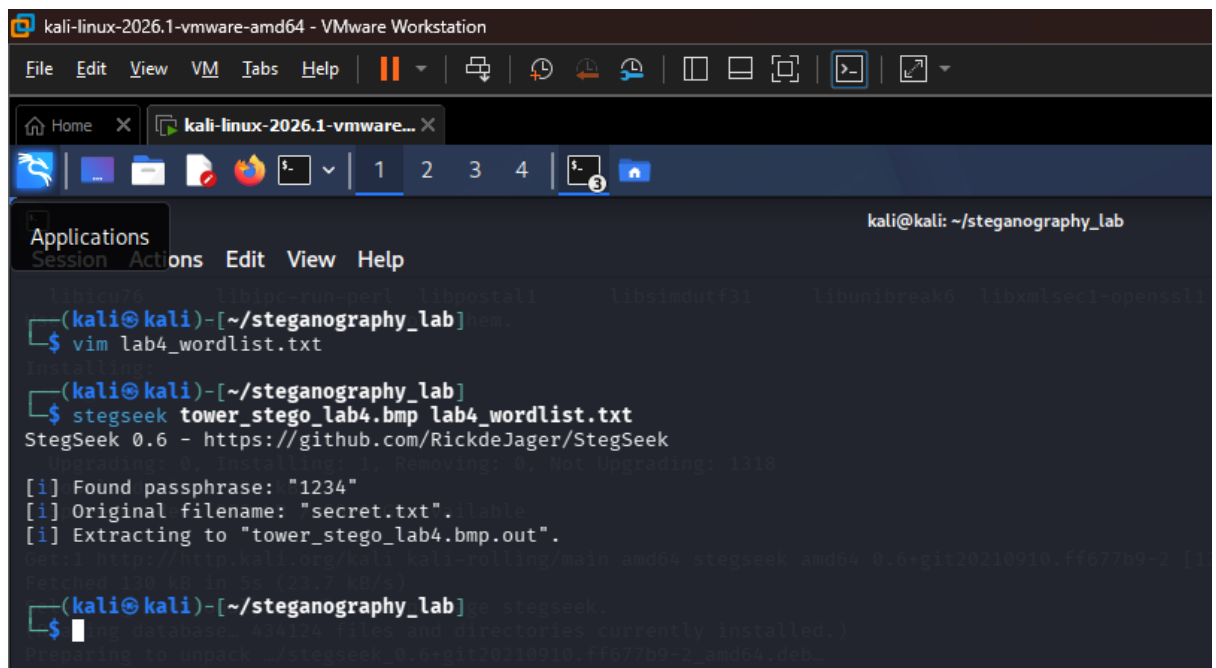
2. **Sha256sum:** We then compare the sha256sum of both the original and the stego file

```
(kali@kali)-[~/steganography_lab]
$ sha256sum _tower_original_image_for_lab.bmp tower_stego_lab4.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db _tower_original_image_for_lab.bmp
4d89ebd54a98e981b416a692da1470a7579e3e9be9072751bd43a26c4e777a45 tower_stego_lab4.bmp

(kali@kali)-[~/steganography_lab]
$
```

Fig 1.10 The comparison between the sha256sum values

3. **Initial bytes using xxd:** The Bytes are the the same

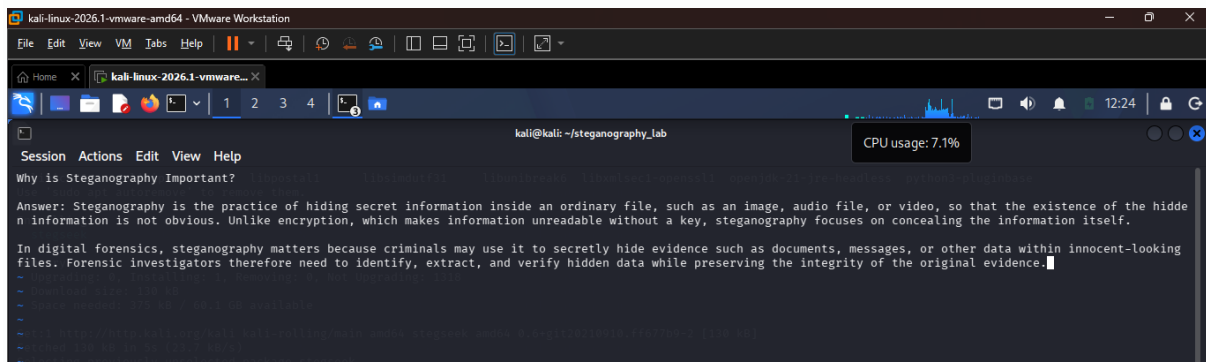


```
kali@kali: ~/steganography_lab
$ vim lab4_wordlist.txt
$ stegseek tower_stego_lab4.bmp lab4_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek
[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "tower_stego_lab4.bmp.out".
$
```

Fig 1.13 Using stegseek to search through the wordlist

Part G - Independent Student Task

Step 1: Creating my own hidden evidence file Dominion_oje_hidden_note.txt



```
kali@kali: ~/steganography_lab
Why is Steganography Important?
Answer: Steganography is the practice of hiding secret information inside an ordinary file, such as an image, audio file, or video, so that the existence of the hidden information is not obvious. Unlike encryption, which makes information unreadable without a key, steganography focuses on concealing the information itself.
In digital forensics, steganography matters because criminals may use it to secretly hide evidence such as documents, messages, or other data within innocent-looking files. Forensic investigators therefore need to identify, extract, and verify hidden data while preserving the integrity of the original evidence.
```

Fig 1.14 Personal hidden note .txt file

Step 2: Embedding it into a Carrier Image **_tower_image_for_lab.bmp** and outputting it into another file **personal_stego_image.bmp**

```
(kali@kali)-[~/steganography_lab]
$ ls
Dominion_Oje_hidden_note.txt  extracted_hash.txt  original_hash.txt  _tower_image_for_lab.bmp  tower_stego_lab4.bmp
extracted                     lab4_wordlist.txt   secret.txt         _tower_original_image_for_lab.bmp  tower_stego_lab4.bmp.out

(kali@kali)-[~/steganography_lab]
$ steghide embed -ef Dominion_Oje_hidden_note.txt -cf _tower_image_for_lab.bmp -sf Personal_stego_image.bmp -p mayord669@
embedding "Dominion_Oje_hidden_note.txt" in "_tower_image_for_lab.bmp" ... done
writing stego file "Personal_stego_image.bmp" ... done

(kali@kali)-[~/steganography_lab]
$
```

Fig 1.13 Embedding my personal secret message into a carrier image

Step 3: Extracting it again.

```
(kali@kali)-[~/steganography_lab]
$ mkdir -p Personal_extracted
steghide extract -sf Personal_stego_image.bmp -xf Personal_extracted/Personal_extracted_secret.txt -p mayord669@
wrote extracted data to "Personal_extracted/Personal_extracted_secret.txt".

(kali@kali)-[~/steganography_lab]
$
```

Fig 1.13 Extracting it using steghide into a personal_extracted file

Step 4: Next we verify the integrity of the files using their hashes.

```
(kali@kali)-[~/steganography_lab]
$ sha256sum _tower_image_for_lab.bmp Personal_stego_image.bmp
2712d58bfba737ad211a9ed45c39c89ea9f9e3c1be921861e7174b4539726bdb _tower_image_for_lab.bmp
9ee63eb0ba117115a7f2254c89431156fcd30f1ad67484d933899315c8e3dde8 Personal_stego_image.bmp

(kali@kali)-[~/steganography_lab]
$
```

Fig 1.13 checking the hashes of the .bmp files

We can see that both files have different hashes which means the embedding was correctly done.

Now checking for integrity of the hidden file that was embedded and also extracted

```
(kali@kali)-[~/steganography_lab]
$ md5sum Dominion_Oje_hidden_note.txt Personal_extracted/Personal_extracted_secret.txt
b19b7c2745e378a8b9b092db31a8aed8 Dominion_Oje_hidden_note.txt
b19b7c2745e378a8b9b092db31a8aed8 Personal_extracted/Personal_extracted_secret.txt

(kali@kali)-[~/steganography_lab]
$
```

Fig 1.14 checking the integrity of the embedded and extracted files

Step 5: Creating a small wordlist named personal_wordlist.txt containing my chosen password.

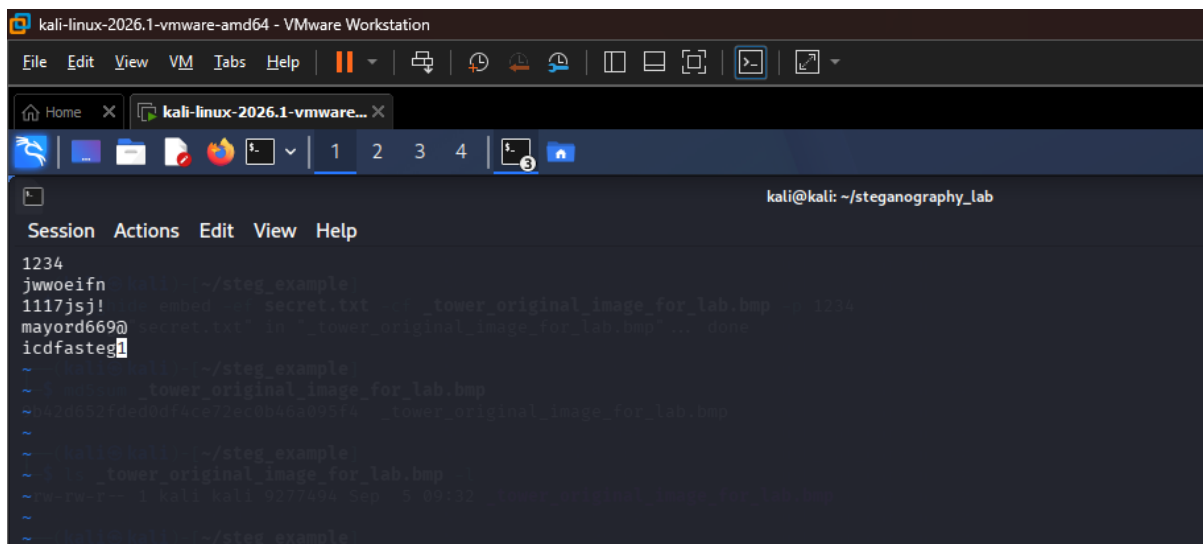


Fig 1.15 created a small wordlist with the password contained

Step 6: Testing the recovery workflow

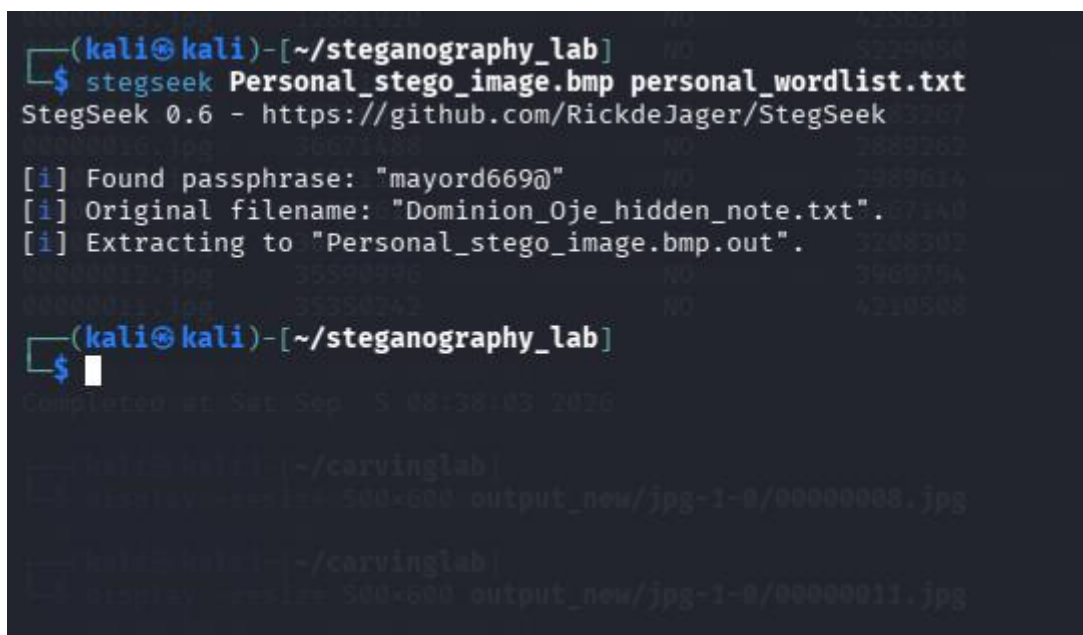


Fig 1.16 Running the steg_image_file against the wordlist

And it showed that the recovery process was successful as the password was found.

In conclusion, the lab successfully demonstrated how steganography can conceal information within ordinary digital files and how forensic tools can be used to **embed, extract, verify, and analyze hidden data**. It highlighted the importance of hashing and careful evidence handling in maintaining the **integrity and reliability of digital forensic investigations**.